PROJECT REPORT

SHA3-256 Hash Accelerator

Design and Verification in Verilog

A hardware core for the SHA-3 hash function, verified in simulation and synthesised with Yosys

Abstract

This project builds and verifies a SHA3-256 hash core in Verilog that produces a digest in 25 clock cycles for a single-block message. The core takes a message in 136-byte blocks, adds the SHA-3 padding itself, and runs the Keccak-f[1600] permutation at one round per clock cycle.

A self-checking testbench compared the core with Python's `hashlib` on 627 messages and found no mismatches. Yosys 0.33 synthesised the design into 15,387 generic cells, 1,609 of them flip-flops, and its design check reported no problems. The code, testbench and results are published in the GitHub repository `sha3-256-accelerator`.

1. Introduction and Objectives

A hash function turns a message of any length into a short, fixed-size fingerprint. Hash functions are used in digital signatures, secure boot and data-integrity checks. SHA-3, standardised by NIST in FIPS 202, is the newest family of NIST hash functions, and a hardware core takes this work off the processor.

The objectives of this project are as follows:

1. To build a SHA3-256 core in Verilog with valid/ready handshakes on both the input and the output.
2. To do the padding inside the core, including the case where the last block is exactly full.
3. To verify the core against a trusted software model with directed, corner-case and random tests.
4. To synthesise the design to confirm that it maps to real logic, and to report its size.

The scope is SHA3-256 only, at one round per clock cycle. Real chip area, clock speed and power are outside the scope; they would need a physical design flow.

2. Background: How SHA-3 Works

SHA-3 is a sponge: it absorbs the message into a large internal state, scrambles that state, and then squeezes the hash out of it. The state is 1600 bits, arranged as 25 lanes of 64 bits. For SHA3-256, 1088 bits (136 bytes) of the state take in message bytes, and the other 512 bits never touch the message directly.

The scrambling step is the Keccak-f[1600] permutation: 24 rounds, each made of five steps.

Step	What it does
theta	Mixes each column with its two neighbouring columns using XOR and a one-bit rotation
rho	Rotates every lane by a fixed amount
pi	Moves the lanes to new positions
chi	The only non-linear step: combines each lane with two neighbours in its row using AND and NOT
iota	XORs a round constant into one lane, so no two rounds are the same

A message rarely fills a whole number of 136-byte blocks, so SHA3-256 pads it. The padding adds the byte `0x06`, then zeros, and sets the last byte of the block to `0x80`. If only one byte is left free, both marks share it and the byte becomes `0x86`. If the message is an exact multiple of 136 bytes, the padding needs a whole extra block. The digest is the first 256 bits of the state after the last block.

3. Design and Architecture

The core has two parts: a data path that holds and scrambles the 1600-bit state, and a three-state control machine that decides when to take a block, when to run rounds and when the digest is ready.

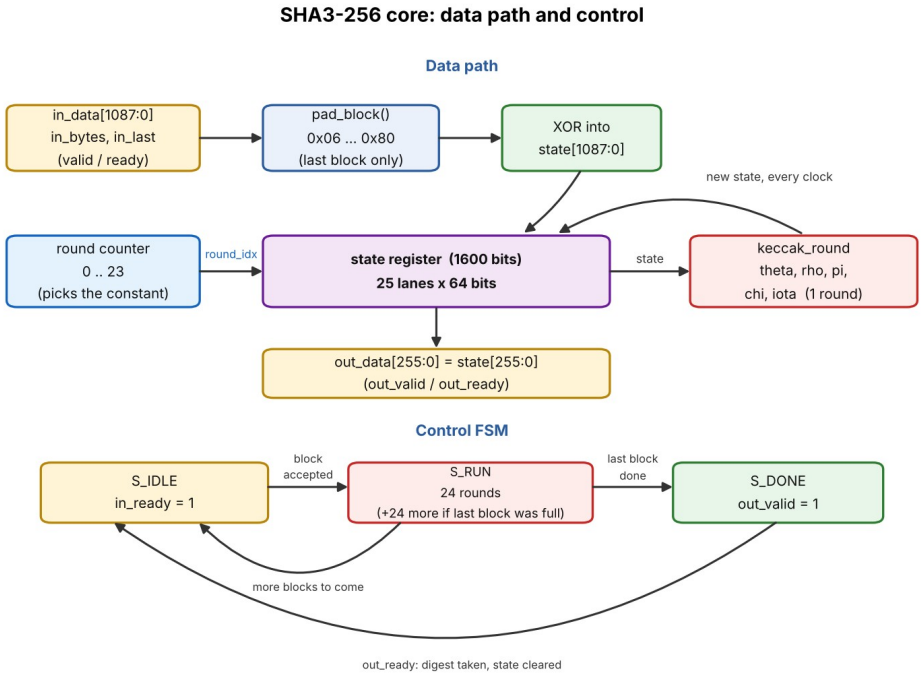


Figure 1: Data path and control state machine of the SHA3-256 core.

1. Data path: A block arrives on `in_data`. The `pad_block` function pads it if it is the last block, and the result is XORed into the low 1088 bits of the state register. The state then passes through one `keccak_round` per clock cycle, and a round counter picks the round constant. The digest is the first 256 bits of the state.

2. Control state machine: it has three states, as shown below.

State	What it means	How it moves on
S_IDLE	`in_ready` is high and the core waits for a block	Accepts a block when `in_valid` is high, then goes to S_RUN
S_RUN	Runs the 24 rounds, one per clock	After round 23, returns to S_IDLE if more blocks are due, or goes to S_DONE after the last block
S_DONE	`out_valid` is high and the digest is ready	When `out_ready` is high, clears the state and returns to S_IDLE

If the last block is exactly 136 bytes, S_RUN absorbs one extra padding block after round 23 and runs 24 more rounds before going to S_DONE.

3. Interface: The input side has `in_valid`, `in_ready`, `in_data[1087:0]`, `in_bytes` (136 unless it is the last block) and `in_last`. The output side has `out_valid`, `out_ready` and `out_data[255:0]`. Byte `i` of a block is at `in_data[8*i +: 8]` and byte `j` of the digest is at `out_data[8*j +: 8]`.

4. Timing: The core takes one clock cycle to accept a block and 24 to run the rounds, so the digest is visible 25 clock cycles after a single-block message is accepted.

Block	Function
pad_block	Adds the SHA-3 padding to the last block
State register	Holds the 1600-bit state
keccak_round	Does one round of theta, rho, pi, chi and iota
Control FSM	Moves between S_IDLE, S_RUN and S_DONE and counts rounds
out_data	First 256 bits of the state, the final digest

4. Implementation

The design is two Verilog modules, a testbench and a Python reference model, all simulated with free tools on Ubuntu under WSL2.

File	Purpose
`rtl/keccak_round.v`	One Keccak round, purely combinational
`rtl/sha3_256.v`	Top module: control state machine, padding, valid/ready handshakes
`tb/tb_sha3_256.v`	Self-checking testbench
`model/keccak_ref.py`	Readable Python reference model
`model/gen_vectors.py`	Generates the 627 test vectors, with `hashlib` as the golden model
`model/check_model.py`, `model/rtl_mirror.py`	Python-only checks of the model and of the Verilog tables

Tools used: Verilog HDL, Icarus Verilog 12 for simulation, GTKWave for waveforms, Python 3 for the model and the vectors, and Yosys 0.33 for synthesis. Three design decisions are worth noting:

- **Byte and lane order.** Byte i of the state sits at bits `[8*i +: 8]`, so lane k is bits `[64*k +: 64]`. A 136-byte block can then be XORed straight into the low 1088 bits of the state.
- **Tables checked automatically.** The 25 rotation offsets and 24 round constants in the Verilog are read out by a script and compared with the verified Python model, so a typing mistake in a table cannot go unnoticed.
- **Padding inside the core.** The core pads the last block itself and ignores any bytes beyond the stated message length. The testbench fills those unused bytes with random data to prove this.

5. Verification

All 627 test messages produced the correct digest, with no other errors, when run in Icarus Verilog 12. The expected digests come from Python's `hashlib`, and four of them were also checked against published values. The test messages cover:

- The standard published test messages: the empty message, "abc", and the 448-bit and 896-bit messages.
- Lengths around the 136-byte block edge (134, 135, 136, 137, 271 to 273, and 407 to 409 bytes), where padding is most likely to go wrong.
- All-zero and all-one blocks, and a 1000-byte message that needs eight blocks.
- 600 random messages of 0 to 420 bytes.

The testbench also stresses the handshakes and the interface:

- Random idle gaps of 0 to 3 clock cycles before each input block.
- Random back-pressure on the output, so the digest waits for `out_ready`.
- Random garbage in the unused bytes of the last block, which the core must ignore.
- A monitor that fails the test if `out_valid` or `out_data` changes while `out_ready` is low.

The simulation log is saved in the repository as `docs/sim_log.txt`. In the waveform, the state register `st` shows `00` (waiting), `01` (running) and `10` (digest ready), and the round counter runs from 0 to 23, one round per clock.

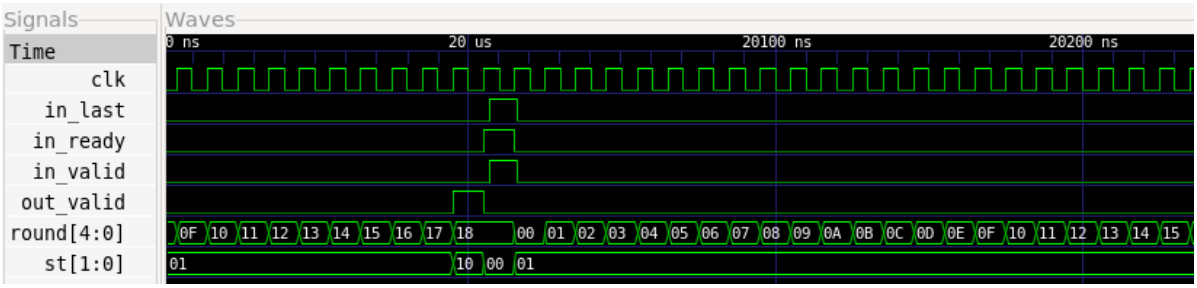


Figure 2: Simulation waveform (GTKWave). The round counter runs from 0 to 23, one round per clock, while st moves through running and digest-ready.

6. Synthesis

Yosys 0.33 turned the design into 15,387 generic cells and reported no problems in its design check. The run used a flattened design with the top module `sha3_256` and Yosys's generic gate library. The full report is saved as `docs/synth_report.txt`.

Cell type	Count	Cell type	Count
XOR	5,082	XNOR	819
ANDNOT	4,212	OR	685
MUX	2,482	NOT	270
Flip-flops (DFFE)	1,609	ORNOT	80
AND	63	NOR	56
NAND	29	**Total**	**15,387**

The 1,609 flip-flops match the design exactly: 1,600 state bits plus 9 control bits (2 for the state machine, 5 for the round counter, 1 for `last_r` and 1 for `extra_r`). The large number of XOR cells comes from the theta and chi steps of the round. These are generic gate counts before mapping to any real technology. They show the size of the design, but not chip area, clock speed or power.

7. Results Summary

Result	Value	How it was obtained
Latency, one-block message	25 clock cycles	Measured in the Verilog testbench
Latency, last block exactly 136 bytes	49 clock cycles	From the Python model of the state machine; not measured in the Verilog simulation
Test messages passed	627 of 627	Verilog testbench against `hashlib` and published digests
Yosys design check	0 problems	Yosys 0.33
Size after synthesis	15,387 generic cells, 1,609 flip-flops	Yosys 0.33, generic gate library

8. Limitations and Future Work

Limitations

- No real area, clock speed or power figures, because the design was not run through a physical design flow such as OpenLane.
- Simulation only: the core has not been run on an FPGA or in silicon.
- The 49-cycle figure for a full last block comes from the Python model, not from the Verilog simulation.
- Only SHA3-256 is supported. The other SHA-3 sizes use a different block size.
- The tests use `hashlib` and four published messages. They are not the complete NIST test suite.

Future work

- Run OpenLane to get chip area, timing and a layout.
- Compute two rounds per clock cycle, which would roughly halve the latency at the cost of a longer logic path, and compare the two versions.
- Add the other SHA-3 sizes (224, 384 and 512).

9. Conclusion

The project delivered a SHA3-256 hash core that gives the correct digest on all 627 test messages and takes 25 clock cycles for a single block. Yosys maps it to 15,387 generic cells with no design-check problems. The test setup, which combines a trusted software model, published test messages, random handshake timing and a stability monitor, is as important a result as the design itself. The next step would be a physical design run to obtain real area and timing.

References

1. NIST, FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions, 2015.

2. Test vectors for SHA-1, SHA-2 and SHA-3, https://di-mgt.com.au/sha_testvectors.html (opened 6 November 2026).

3. Python Software Foundation, hashlib module, used as the golden reference model.

4. Project repository: sha3-256-accelerator on GitHub.